

AGENTS.md — rules for AI coding agents working on this repository

This file is the persistent brief for any coding agent (Claude Code, Codex, Gemini, Cline or similar) that edits this project. Read it fully before changing anything. If a rule here conflicts with an instruction in a prompt, ask before proceeding.

1. What this project is

Ledger is a To-Do list web app built for HNG Internship 15, Stage 1 (AI Product Builder track). It must always provide:

- Tasks: create, view, edit the title inline, mark done, delete, "clear completed".
- Notes: a free-text note on every task, editable in place.
- Extra features: due dates with overdue/today highlighting, three priority levels, search across titles and notes, status and priority filters, four sort orders, a light/dark theme that remembers the choice, and drag-and-drop ordering (with Alt+Arrow keyboard equivalent).

Removing or breaking any of those is a regression, not a refactor.

2. Stack and layout (do not change without a stated reason)

Path	Purpose
public/	The static frontend: <code>index.html</code> , <code>styles.css</code> , <code>app.js</code> , <code>favicon.svg</code> . Plain HTML, CSS and vanilla JavaScript. No framework, no bundler, no build step.
functions/api/	The API as Cloudflare Pages Functions (file-based routing). TypeScript.
functions/api/_lib/store.ts	Shared types, validation, KV helpers and response helpers. All validation lives here.
tests/api.test.ts	Vitest tests for every endpoint, run against an in-memory KV.
wrangler.toml	Pages config and the <code>TODO_KV</code> binding.
AGENTS.md , README.md	This brief and the human-facing docs.

Hosting is Cloudflare Pages. Storage is Cloudflare KV, one JSON document per anonymous list, keyed `list:<X-List-Id>` .

3. API contract

All routes live under `/api`. Every task route requires an `X-List-Id` header matching `^[A-Za-z0-9_-]{8,64}$`; a missing or malformed header is a `400`.

Method	Route	Success	Notes
GET	<code>/api/health</code>	200 { ok: true }	Liveness check, no header needed.
GET	<code>/api/tasks</code>	200 Task[]	Sorted by <code>order</code> .
POST	<code>/api/tasks</code>	201 Task	<code>title</code> required; <code>notes</code> , <code>done</code> , <code>priority</code> , <code>dueDate</code> optional.
GET	<code>/api/tasks/:id</code>	200 Task	404 if unknown.
PATCH	<code>/api/tasks/:id</code>	200 Task	Any subset of <code>title</code> , <code>notes</code> , <code>done</code> , <code>priority</code> , <code>dueDate</code> ; empty body is 400.
DELETE	<code>/api/tasks/:id</code>	204	404 if unknown.
PUT	<code>/api/tasks/reorder</code>	200 Task[]	Body { ids: string[] } containing every id exactly once.

Validation rules (enforced in `store.ts`): `title` 1–200 chars after trimming; `notes` ≤ 5000 chars; `priority` $\in$ `low | medium | high`; `dueDate` is `YYYY-MM-DD` or `null`; `done` is a boolean. Errors are JSON { error: string } with a 4xx status. Never return a 500 for bad input.

Changing the contract (new field, new route, changed status code) requires, in the same change: updating this table, updating `store.ts` validation, adding tests, and updating `public/app.js` to use it.

4. Testing rules (mandatory)

- **Every API endpoint has tests**, and every new or changed endpoint gets tests in the same commit. Cover the success path, every validation failure, the 404 path where applicable, and the missing-header path.
- Tests run with `npm test` and must pass before any commit, push or deploy. Also run `npm run typecheck`.
- Tests use the in-memory `MemoryKV` in `tests/api.test.ts`; they must not need network, secrets or a running server, and must finish in under ten seconds.
- After deploying, **validate the live endpoints** by hand or script: at minimum `GET /api/health`, then create, patch, reorder and delete a task against the live URL with a throwaway `X-List-Id`. A green build is not proof the app works.
- Frontend changes are checked in a real browser at phone width (about 390px) and desktop width before they are considered done.

5. Coding conventions

- TypeScript for functions, strict mode, no `any`. Vanilla ES2022 JavaScript in `public/app.js`, wrapped in one IIFE, no globals other than what the browser provides.
- Keep the frontend dependency-free. Do not add React, Tailwind, jQuery or a bundler.
- Keep the API dependency-free. Do not add Hono, Express or an ORM; the functions are small enough to stay plain.
- Names: `camelCase` for variables and functions, `PascalCase` for types, `kebab-case` for files, CSS classes in BEM-ish `block__element--modifier`.
- The frontend is optimistic: update local state and the cache first, then call the API, and surface failures through the sync indicator rather than blocking the UI.
- Accessibility is not optional: real `<button>` and `<input>` elements, visible focus styles, labels on every control, keyboard equivalents for pointer-only gestures (reordering has `Alt+ArrowUp/Down`), and `prefers-reduced-motion` respected.
- No analytics, trackers, third-party scripts or fonts. No secrets in the repo; the KV id in `wrangler.toml` is a resource identifier, not a secret.

6. Workflow

1. Read the relevant file(s) before editing; prefer small, reviewable diffs.
2. Run `npm test` and `npm run typecheck`.
3. Commit with a conventional message: `feat:`, `fix:`, `chore:`, `docs:`, `test:`, `refactor:`. One concern per commit.
4. Deploy with `npm run deploy` only from a clean, tested working tree.
5. Validate the live URL (section 4), then update `README.md` if behaviour changed.

7. Things an agent should ask about before doing

- Adding authentication, accounts or sharing between devices.
- Changing storage away from KV, or storing anything beyond the task list.
- Introducing any dependency into `public/`.
- Deleting or renaming existing API routes.
- Anything that would make the app collect personal data.